

DESAFIO TÉCNICO

Desenvolvedor(a) Pleno Full Stack

Tribe Lyceum - Techne

Prazo de entrega	3 dias corridos após a entrevista com a Werecoiter
Stack obrigatória	Backend em Java com Spring Boot. Frontend em Angular, JavaScript, TypeScript ou framework web equivalente.
Domínio do desafio	Sistema acadêmico de matrículas, alinhado à organização em camadas, testabilidade, documentação e evolução sustentável de produto legado.
Entrega final	Link do repositório com README completo, instruções de execução, testes e principais decisões técnicas.

Bem-vindo(a) ao desafio técnico

Este desafio faz parte do processo seletivo para Desenvolvimento Pleno. Mais do que validar apenas código funcional, queremos entender sua autonomia, organização, testabilidade, clareza de responsabilidades e capacidade de entregar uma aplicação sustentável.

01 Instruções gerais

Leia esta seção antes de iniciar a implementação.

1. Você deve desenvolver a solução e disponibilizar o código em um repositório acessível para avaliação.
1. O prazo de entrega é de 7 dias corridos a partir do envio do desafio.
2. Ao finalizar, envie o link do repositório com README contendo instruções de execução, testes, tecnologias utilizadas, principais decisões técnicas e uso de IA.
3. A solução precisa ser executável de forma reproduzível. Para este nível, é esperado Docker Compose com banco de dados e instruções claras para execução local.
4. A entrevista técnica, quando houver continuidade no processo, será baseada no código entregue. Você deverá explicar decisões, regras críticas, testes e eventual uso de IA.

► Uso de IA

O uso de ferramentas de IA é permitido e incentivado. Informe no README quais ferramentas utilizou, em quais partes do projeto, quais decisões revisou manualmente e quais trechos considera mais críticos.

O uso de IA não será avaliado negativamente. O ponto crítico é a sua capacidade de explicar o código, as decisões tomadas e os trade-offs assumidos.

02 Desafio funcional

Base comum do desafio técnico.

Desenvolva uma solução para gestão de matrículas acadêmicas. A solução deve permitir cadastrar alunos, cursos, disciplinas e turmas, além de controlar matrículas com regras de negócio claras.

► **Entidades sugeridas**

- Aluno
- Curso
- Disciplina
- Turma
- Matrícula

► **Regras de negócio mínimas**

- Um aluno só pode ser matriculado em turmas abertas.
- Uma turma possui limite de vagas.
- Um aluno não pode se matricular duas vezes na mesma turma.
- Uma matrícula possui status: PENDENTE, CONFIRMADA ou CANCELADA.
- Ao confirmar uma matrícula, a vaga da turma deve ser consumida.
- Ao cancelar uma matrícula confirmada, a vaga deve ser liberada.
- Deve haver consulta de matrículas por aluno e por turma.

03 Requisitos técnicos obrigatórios para Pleno

Organização, testabilidade e entrega sustentável.

Objetivo: avaliar autonomia, organização, testabilidade, clareza de responsabilidades e capacidade de entregar uma aplicação sustentável.

Área	Expectativa
Backend	Java com Spring Boot; API REST funcional; separação clara entre controller, service/application, domain/model, repository/persistence e DTOs.
Funcionalidades	Cadastro, edição, listagem e exclusão de alunos, cursos, disciplinas e turmas; matrícula de aluno em turma; consultas de matrículas por aluno e por turma.
Persistência	Banco de dados relacional usando JPA/Hibernate.
Migrations	Controle de evolução do banco com Flyway ou Liquibase.
Validações	Validação de entrada nas APIs, com mensagens claras e consistentes.
Tratamento de erros	Tratamento padronizado de erros, evitando respostas genéricas e facilitando diagnóstico.
Testes automatizados	Testes cobrindo as principais regras de matrícula, especialmente limite de vagas, matrícula duplicada, confirmação e cancelamento.
Testes de integração/API	Testes de integração ou testes de API para validar fluxos principais e persistência.
Ambiente	Docker Compose com banco de dados e instrução confiável de execução local.
Documentação da API	Swagger/OpenAPI disponível para consulta e teste dos endpoints.
Frontend	Frontend estruturado com componentes, telas separadas, tratamento de erros e consumo organizado da API.
README	README com execução, testes, decisões técnicas, tecnologias usadas e uso de IA.

► Resumo do que esperamos encontrar no repositório

Entrega obrigatória	O que será observado
• API REST funcional em Spring Boot. • CRUD das entidades principais. • Fluxo completo de matrícula. • Banco relacional com JPA/Hibernate. • Migrations com Flyway ou Liquibase. • Docker Compose com banco de dados. • Swagger/OpenAPI configurado.	• Separação clara de camadas. • DTOs e validações de entrada. • Tratamento padronizado de erros. • Testes das regras críticas. • Testes de integração ou API. • Frontend organizado e consumindo a API. • README claro, completo e reproduzível.

Nota sobre escopo Pleno

Não é obrigatório criar múltiplos serviços, mensageria real ou arquitetura distribuída. Para este nível, priorize uma solução bem organizada, testável, clara e executável. Complexidade sem justificativa técnica será avaliada negativamente.

04 README esperado

O README é parte essencial da avaliação.

- Como rodar o projeto localmente.
- Como subir o banco de dados com Docker Compose.
- Como executar os testes automatizados.
- Como acessar a documentação Swagger/OpenAPI.
- Quais tecnologias foram usadas.
- Quais foram as principais decisões técnicas.
- Como a regra de vagas foi protegida.
- Como foram testadas as regras críticas de matrícula.
- Quais limitações conhecidas existem na solução.
- Quais ferramentas de IA foram utilizadas, em quais partes, o que foi revisado manualmente e quais trechos são mais críticos.

05 Critérios críticos de avaliação

Itens que podem impedir a continuidade no processo.

Tema	Ponto crítico
Execução	Projeto não roda ou não há instrução clara de execução.
Stack	Backend não foi desenvolvido em Spring Boot.
Persistência	Não há persistência de dados em banco relacional.
Regras de negócio	Regra de matrícula não está implementada ou a regra de vagas pode ser quebrada facilmente.
Camadas	Não há separação clara de camadas ou o código está concentrado em controllers.
Testes	Não há testes para regras críticas de matrícula.
Ambiente	Não há Docker Compose ou instrução equivalente confiável para executar o banco de dados.
Erros	Não há tratamento adequado e padronizado de erros.
Documentação	O README não explica como executar, testar e validar a solução.
Entrevista	O candidato não consegue explicar as decisões técnicas, o código entregue ou o uso de IA.

06 Diferenciais

Itens não obrigatórios, mas valorizados para o nível Pleno.

- CI executando build e testes.
- Paginação e filtros nas consultas.
- Logs estruturados.
- Eventos internos de domínio, mesmo sem mensageria externa.
- Boa organização do frontend.
- Testes com cenários de erro e borda.
- Tratamento transacional consistente nas regras de matrícula.
- Uso de IA bem documentado e revisado criticamente.

07 Como sua entrega será analisada

Dimensões de avaliação técnica.

A avaliação buscará consistência entre funcionalidade entregue, organização técnica, testes, documentação e capacidade de explicar as escolhas realizadas.

Dimensão	O que será observado
Funcionalidade entregue	Fluxos principais funcionando e aderentes às regras de negócio.
Organização do backend	Separação de responsabilidades, clareza de camadas e modelagem.
Regras de negócio	Consistência da matrícula, controle de vagas e cancelamento.
Testes	Cobertura das regras críticas e qualidade dos cenários.
Frontend	Organização, fluxo de uso e tratamento de erros.
Documentação	README, Swagger/OpenAPI e explicação das decisões técnicas.
Arquitetura/desacoplamento	Organização interna, baixo acoplamento e escolhas coerentes para uma aplicação Pleno.

► Sinais qualitativos valorizados

- Clareza de raciocínio e simplicidade nas escolhas.
- Capacidade de explicar o próprio código.
- Cuidado com erros, validações e transações.
- Consistência entre README, arquitetura e implementação.
- Pragmatismo: nem código improvisado, nem complexidade gratuita.
- Autonomia para tomar boas decisões sem perder clareza e manutenção.

08 Entrevista técnica após a triagem

A conversa será baseada no código entregue.

Caso sua entrega avance para a etapa técnica, a entrevista buscará confirmar autoria, profundidade técnica e maturidade nas decisões. Exemplos de pontos que poderão ser explorados:

- Mostre o fluxo de matrícula de aluno em turma.
- Onde está protegida a regra de limite de vagas?
- O que impede um aluno de se matricular duas vezes na mesma turma?
- O que acontece ao confirmar e ao cancelar uma matrícula?
- Como você testou as regras críticas?
- Como você estruturou controller, service, domínio, repository e DTOs?
- Como o frontend consome a API e trata erros?
- Qual decisão você tomaria diferente com mais tempo?
- Que parte foi feita com ajuda de IA?
- Explique um trecho crítico sem consultar documentação.

Boa sorte!

Valorize clareza, consistência, simplicidade e justificativa técnica.